B.TECH CCE REVISION · PART 5

Machine Learning, Data Mining and Predictive Analysis

Covers the marksheet subjects Data Mining and Predictive Analysis (and lab), Pattern Recognition, and the useful parts of Engineering Mathematics III and IV.

The maths that carries weight, how models learn and how to tell if they work, the classic algorithms, data mining and pattern recognition, neural networks, and how LLMs and agents fit on top.

Isagi · November 2026 · Written from general knowledge of the field as of late 2026; check tool versions, prices and vendor specifics against current documentation.

How to use this chapter

This is the module closest to agentic AI and to any product that scores, ranks or predicts. Predictive analysis was rated weak, and pattern recognition is the subject to strengthen. The Part 1 primer already explained LLMs at high level; this part builds the foundation underneath, so terms like overfitting, precision, embeddings and gradient descent stop being vocabulary and become tools you can reason with. The maths is limited to what changes how you think. No derivations.

> **How to read it.** Sections 1–4 are the core: if you only do part of this chapter, do those and Lab 1. The labs use scikit-learn, which installs with `pip install scikit-learn pandas`.

Contents

1. The maths that carries weight
2. What machine learning is
3. Generalisation: train, validation, test, overfitting
4. Evaluation: what "accurate" should mean
5. Classic models and when to use them
6. Features and data quality
7. Data mining and pattern recognition
8. Neural networks and how they learn
9. From neural networks to LLMs
10. Agents, from an ML point of view
11. Labs
12. Self-check questions and answers
13. Resources

1. The maths that carries weight

Three areas: linear algebra (data is vectors), probability and statistics (data is noisy), and calculus (learning is going downhill).

Linear algebra

- A **vector** is a list of numbers: a student described by `[sessions, avg_score, days_active]`, or a sentence turned into 1,536 numbers by an embedding model. Geometrically it's a point or an arrow in space.
- A **matrix** is a table of numbers; a dataset is a matrix (rows = examples, columns = features). Multiplying a vector by a matrix is a **transformation**: it rotates, stretches or projects the vector into another space. A neural network layer is exactly this, followed by a simple non-linear function.
- The **dot product** of two vectors measures how much they point the same way. **Cosine similarity** (the dot product of normalised vectors) is how "semantic similarity" is computed in RAG: close to 1 means similar meaning.
- **Dimensionality**: real data lives in many dimensions, and intuition from 2D and 3D partly breaks (in high dimensions, most points are far from each other). That's why dimensionality reduction (Section 7) is useful.

Probability and statistics

- **Distributions** describe how values spread. The **normal** (bell curve) shows up when many small effects add up; incomes, city sizes and social media followers follow skewed, long-tailed distributions where the **mean** is dragged up by a few huge values and the **median** is more honest.
- **Variance and standard deviation** measure spread. Two classes with the same average score can look completely different.
- **Conditional probability and Bayes' rule**: $P(A|B) = P(B|A) \times P(A) / P(B)$. The classic example: a test that's 99% accurate for a condition that affects 1 in 1,000 people. Of every 100,000 people tested, ~100 have it (99 test positive) and ~99,900 don't (999 false positives). So a positive result means only about a 9% chance of having it. **Base rates matter**, and ignoring them is the most common reasoning error in fraud detection, medical tests and spam filtering.
- **Sampling and hypothesis testing**: is the difference between two versions real, or noise? A **p-value** is the probability of seeing a difference at least this big if there were truly no difference. A **confidence interval** gives a range of plausible values. With small samples, big-looking differences are often noise; with huge samples, trivially small differences become "significant". Always ask about effect size too.
- **Correlation is not causation.** Students who practise more score higher, but motivated students both practise more and score higher. Only experiments (A/B tests) or careful causal methods separate the two.

Calculus intuition

- A **derivative** is the slope: how much the output changes when you nudge the input.
- A **gradient** is the slope in every direction at once, for functions of many variables. It points uphill.
- **Gradient descent**: to minimise a loss (error), repeatedly step a little in the downhill direction. Training almost every modern model, including LLMs with hundreds of billions of parameters, is this loop at enormous scale.

2. What machine learning is

Instead of writing the rules, you show the computer examples and it fits a function that reproduces them, and hopefully generalises to new cases.

Type	You give it	It learns to	Examples
Supervised	Inputs with correct answers (labels)	Predict the answer for new inputs	Spam or not; score an answer 0–10; predict churn
Unsupervised	Inputs only	Find structure	Cluster students by behaviour; detect anomalies; compress
Self-supervised	Raw data where the "label" comes from the data itself	Predict hidden parts	LLM pretraining: predict the next token
Reinforcement learning	An environment and rewards	Act to maximise reward	Game-playing; RLHF to make LLMs helpful

Within supervised learning:

- **Classification** predicts a category (will churn / won't churn; fluent / needs work).
- **Regression** predicts a number (next week's sessions; a score).

The workflow is the same every time: define the question and the metric → get and clean data → split it → choose a simple baseline → train → evaluate on held-out data → look at the errors → improve features or model → deploy → monitor, because the world changes.

3. Generalisation

A model is only useful if it works on data it hasn't seen. Everything in ML practice protects that.

- **Training set**: the model learns from it. **Validation set**: you use it to choose models and settings. **Test set**: touched once, at the end, to estimate real-world performance. Reusing the test set to make decisions slowly turns it into training data.
- **Cross-validation**: split the data into k folds, train on k–1, test on the remaining one, rotate, average. Gives a steadier estimate on small datasets.
- **Overfitting**: the model memorises the training data, noise included, and fails on new data. Training accuracy high, validation accuracy much lower. **Underfitting**: too simple to capture the pattern; both are low.
- **Bias–variance trade-off**: simple models are consistently wrong in the same way (high bias); complex models are wrong in different ways depending on the sample they saw (high variance). More data reduces variance; better features and more flexible models reduce bias.
- **Regularisation** penalises complexity (large weights, deep trees) so the model prefers simpler explanations. L1 (lasso) also pushes useless features to zero; L2 (ridge) shrinks all weights.

> **Data leakage** is the most expensive mistake in applied ML: information from the answer sneaks into the inputs. Examples: a "days since last login" feature computed after the churn date; normalising using statistics from the whole dataset including the test set; the same student's sessions in both train and test. The model looks brilliant in testing and fails in production. Split by time or by user, and compute every feature only from data available at prediction time.

4. Evaluation

Accuracy alone misleads whenever classes are unbalanced, which is most of the time.

If 95% of users don't churn, a model that always says "won't churn" is 95% accurate and useless. Use the **confusion matrix**:

	Predicted positive	Predicted negative
Actually positive	True positive (TP)	False negative (FN) — missed
Actually negative	False positive (FP) — false alarm	True negative (TN)

Metric	Formula	Question it answers
Precision	TP / (TP + FP)	Of those we flagged, how many were right?
Recall (sensitivity)	TP / (TP + FN)	Of all real positives, how many did we catch?
F1	harmonic mean of the two	One number balancing both
ROC-AUC	area under the true-positive vs false-positive curve	How well does the model rank positives above negatives, across all thresholds?
PR-AUC	area under precision vs recall	Like ROC-AUC, but more honest when positives are rare

Precision vs recall is a business choice, set by moving the decision threshold. A fraud filter that blocks payments needs high precision (false alarms anger customers). A screening test for something serious needs high recall (misses are costly). A coaching nudge ("you seem stuck, want help?") can tolerate false alarms.

For regression: **MAE** (mean absolute error, in the units of the target: "off by 0.6 points on average"), **RMSE** (punishes big misses more), and **R^2** (share of variance explained).

Calibration: when a model says 70%, is it right 70% of the time? Many models rank well but are badly calibrated; if probabilities drive decisions (pricing, triage), check.

Evaluating LLM outputs follows the same logic: build a test set of real inputs with expected properties, score outputs (by rules, human raters, or another model as judge with a clear rubric), and track the score across prompt and model changes. Check **consistency** too: score the same answer five times and look at the spread.

5. Classic models and when to use them

Start simple. On tabular business data, gradient-boosted trees usually win; a well-tuned linear model is often close and easier to explain.

Model	How it works, in a line	Strengths	Weaknesses
Linear regression	Weighted sum of features	Fast, interpretable coefficients	Only straight-line relationships unless you engineer features
Logistic regression	Linear score squashed into a 0–1 probability	Strong, interpretable baseline for classification; well-calibrated	Same linearity limit
k-nearest neighbours	Predict from the k most similar known examples	No training; intuitive	Slow at prediction time; needs scaled features; poor in high dimensions
Naive Bayes	Bayes' rule assuming features are independent	Fast, good for text (spam)	The independence assumption is usually false
Decision tree	A flowchart of yes/no splits	Readable; handles mixed data	Overfits badly on its own
Random forest	Many trees on random subsets, averaged (bagging)	Robust, little tuning	Less interpretable; large
Gradient boosting (XGBoost, LightGBM, CatBoost)	Trees added one by one, each fixing the previous errors	Usually the best on tabular data	More tuning; can overfit
SVM	Finds the widest boundary between classes	Good on small, high-dimensional data	Slow on big data; less used now
Neural networks	Layers of learned transformations	Best on images, audio, text	Data-hungry; opaque; overkill for small tables

Always compare against a baseline: predict the average, the most common class, or "same as last week". A complex model that barely beats a baseline isn't worth its cost.

Interpretability tools: feature importance, partial dependence plots, and **SHAP values** (how much each feature pushed this particular prediction up or down). Useful for debugging and for explaining a score to a user.

6. Features and data quality

Most of the real work is here. A better feature beats a better algorithm almost every time.

- **Feature engineering** turns raw data into signals: from session timestamps, derive "sessions in last 7 days", "days since last session", "share of sessions finished". From a transcript, derive "words per minute", "filler words per minute", "longest pause".
- **Categorical data** needs encoding: one-hot (a column per category) for few categories; target or learned encodings for many.
- **Scaling**: models based on distances or gradients (k-NN, SVM, neural nets, regularised linear models) need features on comparable scales (standardise to mean 0, standard deviation 1). Trees don't care.
- **Missing values**: decide deliberately: drop, fill with median, or add an "is missing" flag (missingness itself is often informative).
- **Imbalanced classes**: use the right metrics (Section 4), class weights, or resampling. Don't celebrate accuracy.
- **Label quality**: if human raters disagree 30% of the time, no model will exceed that ceiling. Measure inter-rater agreement before blaming the model.
- **Drift**: the world changes (new user segments, new exam formats, a new phone microphone), and model performance decays. Monitor input distributions and outcomes after launch.
- **Bias and fairness**: if training data under-represents a group (accents from one region, say), the model will be worse for them. Check performance per segment, not just overall.

7. Data mining and pattern recognition

Finding structure in data without (or before) a specific prediction target.

Clustering

- **k-means**: pick k centres, assign each point to its nearest centre, move each centre to the mean of its points, repeat. Fast and simple; you must choose k, and it assumes round, similar-sized clusters. Use the "elbow" plot or silhouette score to choose k, and then check whether the clusters mean anything.
- **Hierarchical clustering** builds a tree of merges (a dendrogram); cut it at any level.
- **DBSCAN** finds dense regions of any shape and labels sparse points as noise.
- Uses: student segments ("crammers", "steady practisers", "drop-offs"), grouping similar feedback, organising documents by topic (on embeddings).

Association rules

Market-basket analysis: "customers who buy primer and putty also buy emulsion 60% of the time". Three measures: **support** (how often the itemset occurs), **confidence** (how often the rule holds when the left side occurs), and **lift** (how much more likely than chance; above 1 means a real association). The Apriori algorithm finds frequent itemsets efficiently. Applies directly to dealer order data and cross-selling.

Anomaly detection

Find the unusual: fraudulent transactions, a dealer whose orders suddenly drop, a server's error rate spiking, fake accounts. Approaches: statistical thresholds (beyond 3 standard deviations), **isolation forests** (anomalies are easy to isolate with random splits), distance from cluster centres, or prediction error from a model of normal behaviour.

Dimensionality reduction

- **PCA** finds the few directions along which the data varies most and projects onto them: 50 correlated features compressed into 5 components that keep most of the information. Good for visualisation, noise reduction and speed.
- **t-SNE and UMAP** squash high-dimensional data (like embeddings) into 2D for plotting, preserving local neighbourhoods. Useful for eyeballing clusters; distances between far-apart groups on the plot don't mean much.

Pattern recognition

The classical framing: **represent** the input as features, then **classify** it. Speech recognition, handwriting recognition and face detection were built this way for decades, with hand-designed features (MFCCs for audio, edge detectors for images) and statistical classifiers (Gaussian mixture models, hidden Markov models, SVMs). Deep learning replaced hand-designed features with learned ones, but the framing (and terms like feature space, decision boundary, template matching) remains.

Time series and forecasting

Predicting next week's sales or sessions: decompose into **trend**, **seasonality** (weekly, festive season) and noise. Methods range from moving averages and exponential smoothing, through ARIMA, to gradient boosting on lag features ("sales last week, same week last year"). The usual traps: leakage from the future, and ignoring holidays and promotions.

8. Neural networks and how they learn

Stacks of simple units, each computing a weighted sum and a non-linearity, trained end to end by gradient descent.

- A **neuron** computes `output = activation(w₁x₁ + w₂x₂ + … + b)`. The **activation** (ReLU: `max(0, x)`, sigmoid, GELU) adds the non-linearity; without it, stacking layers would collapse to one linear function.
- A **layer** is many neurons in parallel: one matrix multiplication plus activation. A **deep** network stacks many layers, each building more abstract features from the previous ones (edges → shapes → objects in images).
- **Loss function**: a single number measuring how wrong the outputs are (mean squared error for regression, cross-entropy for classification and for LLMs).
- **Backpropagation**: the chain rule from calculus, applied efficiently, gives the gradient of the loss with respect to every weight in one backward pass.
- **Training loop**: take a batch of examples → forward pass → compute loss → backpropagate gradients → update weights by a small step (the **learning rate**) → repeat for many **epochs**. Optimisers like **Adam** adapt the step per weight.
- Too high a learning rate and training diverges; too low and it crawls. **Dropout**, weight decay and early stopping fight overfitting.

Architectures, by data type:

Data	Architecture	Idea
Images	**CNN** (convolutional)	Small filters slide across the image, detecting the same pattern anywhere
Sequences (older)	**RNN / LSTM**	Process tokens one at a time, carrying a memory; slow and forget long contexts
Sequences (now), and increasingly everything	**Transformer**	Attention lets every position look at every other position directly, in parallel
Audio	Spectrogram + CNN or transformer (e.g. Whisper)	Treat sound as an image of frequencies over time

9. From neural networks to LLMs

An LLM is a very large transformer trained to predict the next token, then shaped to be useful.

- **Tokens**: text is split into sub-word pieces (~¾ of an English word each; Hindi and other Indian scripts typically use more tokens per word, which costs more).
- **Embeddings**: each token becomes a vector; meaning lives in the geometry.
- **Attention**: for each token, the model computes how much to "look at" every earlier token and blends their information. Many attention **heads** in parallel, many layers deep.
- **Pretraining**: predict the next token over trillions of tokens of text. This self-supervised task forces the model to learn grammar, facts, reasoning patterns and style.
- **Post-training**: **instruction tuning** on example conversations, then **reinforcement learning** from human or AI feedback, and (for reasoning models) RL on verifiable tasks like maths and code. This turns a text predictor into an assistant.
- **Context window**: how many tokens it can consider at once. Everything the model knows about your situation must be in the context or in its weights.
- **Why hallucination happens**: the model is trained to produce plausible continuations, not to say "I don't know". When the right answer isn't strongly represented, a plausible-sounding wrong one comes out. Mitigations: give it the facts (RAG), ask for citations, allow "unknown", verify.
- **Prompting vs RAG vs fine-tuning**: prompting changes instructions; RAG adds knowledge at query time; fine-tuning changes behaviour and style by further training. For most products: prompt first, add RAG for knowledge, fine-tune only for consistent format or tone at scale.

The Part 1 primer covers APIs, prompting, RAG and tool use in more depth.

10. Agents, from an ML point of view

An agent is an LLM in a loop: it decides an action, a tool executes it, the result goes back into the context, repeat until done.

The ML lens adds three warnings to the Part 1 material:

- **Errors compound.** If each step is 95% reliable, a 10-step task succeeds about 60% of the time (0.95^{10}). Shorter loops, verification steps and human checkpoints matter more than a cleverer prompt.
- **Evaluation is the product.** You can't improve what you don't measure: build task-level evals (did it finish the job correctly?), not just "does the answer look good".
- **Distribution shift applies.** An agent tested on tidy demo tasks meets messy real inputs. Log real traces and review failures regularly.

Typical failure modes: wrong tool or wrong arguments, looping, giving up early, confidently reporting success that didn't happen, and following instructions hidden in tool results (prompt injection, Part 6).

11. Labs

Lab 1. An end-to-end predictive model.

```
import pandas as pd
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import RandomForestClassifier
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import make_pipeline
from sklearn.dummy import DummyClassifier
from sklearn.metrics import classification_report, roc_auc_score

X, y = load_breast_cancer(return_X_y=True, as_frame=True)
X_tr, X_te, y_tr, y_te = train_test_split(X, y, test_size=0.25, stratify=y, random_state=0)

models = {
    "baseline": DummyClassifier(strategy="most_frequent"),
    "logistic": make_pipeline(StandardScaler(), LogisticRegression(max_iter=1000)),
    "forest":   RandomForestClassifier(n_estimators=300, random_state=0),
}
for name, m in models.items():
    m.fit(X_tr, y_tr)
    p = m.predict_proba(X_te)[:, 1]
    print(f"{name:9s} AUC={roc_auc_score(y_te, p):.3f}")
print(classification_report(y_te, models["logistic"].predict(X_te)))
```

On a test run the baseline scored an AUC of 0.50 (no skill), logistic regression 0.995 and the random forest 0.977: the simple, scaled linear model won on this small, clean dataset, which is the point of always trying it first. Then: change the threshold from 0.5 to 0.3 and watch precision and recall trade off; print the forest's `feature_importances_`; try it on a real dataset you care about (Kaggle has sales, churn and student-performance datasets).

Lab 2. Overfitting you can see. Train decision trees with `max_depth` from 1 to 20 on the same split; plot training vs test accuracy. Find where they diverge.

Lab 3. Clustering. Run k-means with k = 2–8 on a dataset (or on your own app's usage data), plot the silhouette score, and describe each cluster in a sentence. Ask whether the clusters would change any decision.

Lab 4. Bayes by hand. Redo the 99%-accurate test example with a 1-in-100 base rate and a 1-in-10,000 base rate. Write one sentence on what this means for any "AI detector" claiming 99% accuracy.

Lab 5. LLM scoring consistency. Take five recorded or written answers. Ask an LLM to score each on a 0–10 rubric five times. Compute each answer's mean and standard deviation. Then tighten the rubric with examples of what a 3, 6 and 9 look like, and repeat. Compare the spreads.

12. Self-check questions

1. **Supervised vs unsupervised?** Supervised learns from labelled examples to predict labels; unsupervised finds structure (clusters, anomalies) in unlabelled data.
2. **Classification vs regression?** Predicting a category vs predicting a number.
3. **Why keep a test set untouched?** Every decision made using it leaks information into the model, so it stops being an honest estimate of performance on new data.
4. **What's overfitting and how do you spot it?** Memorising training data including noise; training performance much better than validation performance.
5. **Why does accuracy mislead on churn prediction?** With rare positives, predicting "no" for everyone scores high accuracy and catches nothing. Use precision, recall, PR-AUC.
6. **Precision vs recall in one line each?** Precision: of what we flagged, how much was right. Recall: of what was really there, how much we caught.
7. **Give an example of data leakage.** Using a feature computed after the event you're predicting, or splitting one user's records across train and test.
8. **Why do base rates matter?** A highly accurate test for a rare condition still produces mostly false positives, because the healthy population is so large.
9. **What model would you try first on a tabular business dataset?** A baseline, then logistic or linear regression, then gradient-boosted trees.
10. **What do support, confidence and lift mean?** How often items appear together; how often the rule holds given the left side; how much more often than chance.
11. **What does PCA do?** Finds the directions of greatest variation and projects data onto a few of them, compressing correlated features.
12. **How does a neural network learn?** Forward pass, measure loss, backpropagation computes gradients, gradient descent nudges weights to reduce loss, repeat.
13. **What does attention do?** Lets each token weigh and pull in information from other tokens, so context shapes meaning.
14. **Why do LLMs hallucinate?** They're trained to produce plausible text, not to know what they don't know; without grounding, plausible-but-wrong outputs appear.
15. **Why do long agent tasks fail more often?** Per-step errors multiply: 95% per step over 10 steps is about 60% overall.

13. Resources

Check these are still available before committing time.

- **3Blue1Brown**: *Essence of Linear Algebra*, *Essence of Calculus* and the *Neural Networks* series (YouTube). The best intuition-builders there are.
- **StatQuest with Josh Starmer** (YouTube). Short, clear videos on every statistics and ML concept in this chapter.
- **James, Witten, Hastie and Tibshirani, *An Introduction to Statistical Learning*** (free PDF, with Python labs). The standard readable textbook for Sections 2–7.
- **Andrew Ng's Machine Learning Specialization** (Coursera/DeepLearning.AI). Structured course with assignments.
- **Andrej Karpathy, *Neural Networks: Zero to Hero*** and his *Intro to Large Language Models* talk (YouTube). Builds backprop and a small GPT from scratch.
- **scikit-learn user guide**. Surprisingly good explanations alongside the code.
- **Anthropic, "Building effective agents"** (engineering blog). Practical agent patterns, for Section 10.